

Table of Contents

Bash Commands	1
Git commands	1

Bash Commands

These commands are used in Git bash terminal to navigate in the Git repo. So practice these commands as these are very much necessary.

<u>Command description</u>	<u>Commands</u>
Print the lists the files in the directory. -al is the option for ls command which asks to show all files including hidden folders. Hidden folder starts with “.Folder/file name”	\$ ls \$ start . (open . in MAC) \$ ls -al <Full path of folderName>
Print Present directory – complete path. ~ called tilde is Home directory.	\$ pwd
Create a local directory using the following command:	\$ mkdir ANY_DIRECTORY_NAME
Navigate to the new directory that created in last command	\$ cd ANY_DIRECTORY_NAME
Navigate to one directory back	\$ cd ..
Create empty new file	\$ touch ANY_FILE_NAME
Remove file	\$ rm ANY_FILE_NAME
Remove directory and its contents	\$ rm -rf ANY_DIRECTORY_NAME
Clears the screen	\$ clear

Git commands

Now that we have initialized our project folder as Git repo, how to use Git and work inside folder?
As we progress in our project, we can create a check point. Check point not only saves a single file, but saves the entire project’s snapshot at that moment. This adding checkpoint is done with set of 2 commands.

\$ git add .

\$ git commit “your message for checkpoint reference.”

The basic workflow

Work on stuff (multiple in project can be modified) - > Add changes (Group changes together in preparation to commit) - > commit (commit everything that was previously added)

“add .”command – add dot says that group all the changes done in current and nested project directory.

“Commit” command will add check point to the history saved in “.Git” folder with a message to refer.

\$ git log

Log command – will show the history with your recent commit on the top.

Add the "demo" to the current directory. Instead of .(dot), we can selectively add file names to the group.	\$ git add demo.txt
Adding : We use the git add command to stage changes to be committed. It's a	

way of telling Git, "please include this change in our next commit"	
Committing: Making a commit is similar to taking a snapshot of a git repository in time. When saving a file, we are saving the state of a single file. With Git, we can save the state of multiple files and folders together. Git Commit : We use the git commit command to actually commit changes from the staging area. When making a commit, we need to provide a commit message that summarizes the changes and work snapshotted in the commit	\$ git commit -m "Adding demo file" If \$ git commit That is without message , opens text editor for enter commit message".
	\$ git log
Use git status constantly to verify what changes are staged and not staged	\$ git status